

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	D.N Shirshika
Reg. No	192425396

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

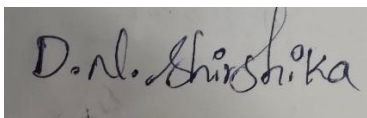
Duration: 30 minutes

Marks: 25

Code of Conduct:

I **D.N Shirshika** __**(192425396)**__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

D.N Shirshika
Name of the student:

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer :

Introduction

Gradient Descent is a widely used optimization algorithm in machine learning that minimizes a loss function by iteratively updating model parameters. The learning rate controls the size of each update step. In this scenario, the learning rate is initially set very high and then gradually decreased during training.

a) Effect of a High Initial Learning Rate

- Allows the algorithm to make large steps toward the minimum.
- Speeds up initial learning and convergence.
- Helps escape shallow regions of the loss surface.
- Reduces training time during early iterations.
- The loss function may fluctuate or even increase.

b) Effect of Gradually Decreasing the Learning Rate

As training progresses, reducing the learning rate:

- Makes parameter updates smaller and more precise.
- Helps the model settle near the optimal solution.
- Reduces oscillations around the minimum.
- Improves convergence accuracy.

Common decay formula:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

Where:

- η_0 = Initial learning rate
- k = Decay constant
- t = Iteration number

c) Conditions Where This Strategy Outperforms a Constant Learning Rate

This strategy performs better when:

- The dataset is large.
- The loss function is convex.
- Fast initial convergence is required.
- High precision is needed near the optimum.
- Training deep learning models with many parameters.

d) Possible Risks and Limitations

- Very high initial learning rate may cause divergence.
- Poorly chosen decay schedule may slow learning too early.
- Learning rate may become too small before reaching the optimum.
- Training may get stuck before finding the best solution.

Diagram

Initial High Learning Rate



Large Parameter Updates



Fast Initial Convergence



Gradual Learning Rate Decay



Smaller Parameter Updates



Stable Optimization



Accurate Convergence

Iterations

Advantages

- Faster training.
- Better optimization performance.
- Improved convergence stability.
- More accurate final solution.

Disadvantages

- Requires careful tuning.
- Incorrect decay schedule may reduce performance.
- Can still diverge if the initial learning rate is extremely large.

Conclusion

Using a high initial learning rate followed by gradual decay is an effective optimization strategy because it combines fast learning in the early stages with precise adjustments near the optimum. This approach often outperforms a constant learning rate by reducing training time and improving convergence accuracy. However, its success depends on selecting appropriate initial and decay values. If the learning rate is too large or decreases too quickly, the model may become unstable or fail to reach the optimal solution. Therefore, learning rate scheduling is an important technique for achieving efficient and reliable optimization in machine learning models.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer:

Introduction

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate unknown parameters of a probability distribution by maximizing the likelihood of observing the given data. For a Gaussian (Normal) distribution with unknown mean μ and known variance σ^2 , MLE is used to find the most probable value of the mean.

a) Derivation of the Maximum Likelihood Estimator for μ

For a Gaussian distribution: $X_i \sim N(\mu, \sigma^2)$

$$\text{Likelihood function: } L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

$$\text{Taking logarithm: } \ell(\mu) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Differentiating with respect to μ and setting equal to zero:

$$\frac{\partial \log L}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^n (x_i - \mu) = 0$$

$$\text{Therefore } \sum_{i=1}^n x_i = n\mu$$

Hence, the MLE estimate is:

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

Result: The MLE of the mean is simply the **sample mean**.

b) Effect of Outliers on the Estimate

Outliers are extreme observations that differ significantly from other data points.

Suppose dataset:

10, 12, 11, 13, 12

Mean:

$$\hat{\mu} = 11.6$$

Adding an outlier:

10, 12, 11, 13, 12, 100

New Mean:

$$\hat{\mu} = 26.3$$

Effects:

- Mean shifts significantly toward the outlier.
- Parameter estimate becomes biased.
- Prediction accuracy may decrease.
- MLE becomes sensitive to extreme values.

c) Effect of Increasing Sample Size

As sample size increases:

$$n \uparrow$$

Properties:

- Estimate becomes more accurate.
- Variance of estimator decreases.
- Sampling error reduces.
- Estimate approaches the true population mean.

According to the Law of Large Numbers:

$$\hat{\mu}_{MLE} \rightarrow \mu$$

as

$$n \rightarrow \infty$$

Benefits:

- Better generalization.
- More stable parameter estimation.
- Improved model reliability.

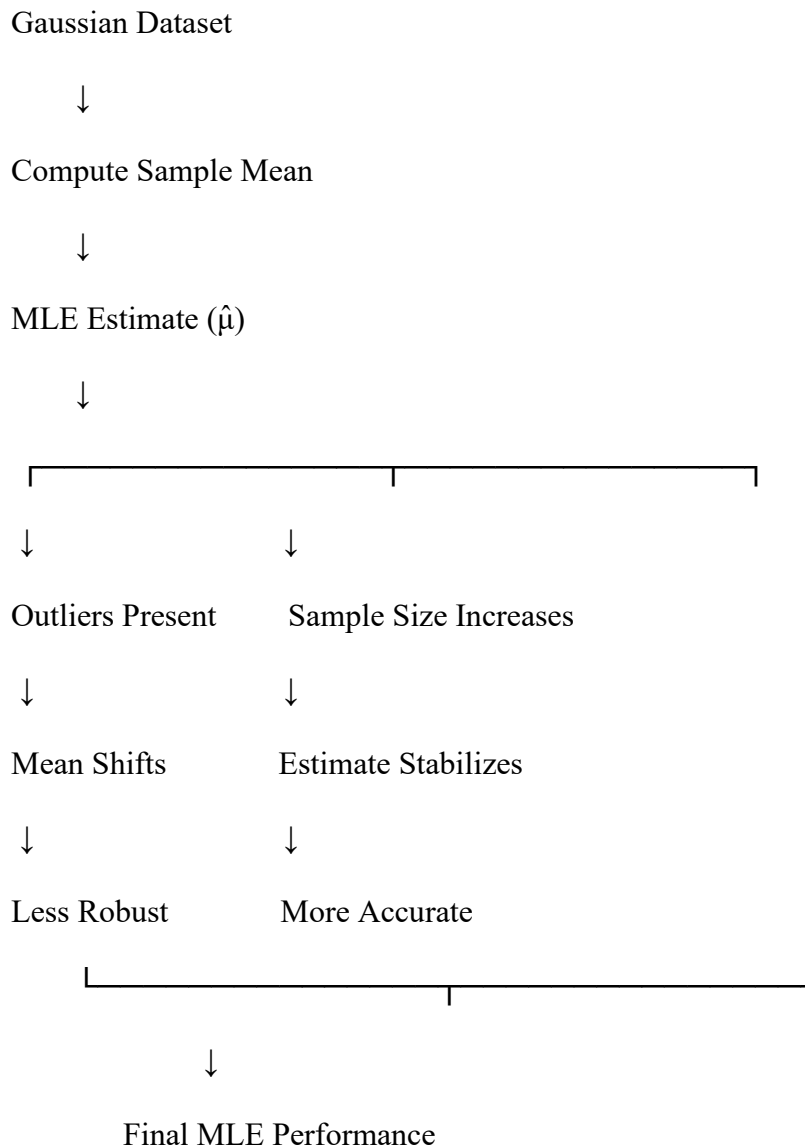
d) Implication on Robustness of MLE

Robustness refers to the ability of an estimator to resist the influence of outliers.

MLE for Gaussian mean:

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum x_i$$

Diagram



Conclusion

Maximum Likelihood Estimation is a powerful method for estimating the mean of a Gaussian distribution. By maximizing the likelihood function, the MLE of the mean is obtained as the sample mean. As the sample size increases, the estimate becomes more accurate and converges to the true population mean, demonstrating the consistency of MLE. However, because the sample mean is highly sensitive to extreme values, the presence of outliers can significantly distort the estimate. This indicates that while MLE is efficient and accurate for normally distributed data, it is not highly robust to outliers. Therefore, robust estimation techniques may be preferred when datasets contain significant noise or extreme observations.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Introduction

In this scenario, the dataset contains **10,000 features** but only **500 training samples**. This is a high-dimensional learning problem where the number of features is much larger than the number of observations. Such datasets are prone to overfitting and poor generalization. Therefore, careful model selection and preprocessing are required to build an effective classification algorithm.

a) Challenges in Training the Model

1. Curse of Dimensionality

- Feature space becomes extremely large.
- Data points become sparse.
- Learning meaningful patterns becomes difficult.

2. Overfitting

- Model may memorize training data.
- High training accuracy but poor testing accuracy.

3. Computational Complexity

- Increased memory requirements.
- Longer training time.
- More complex optimization.

4. Irrelevant Features

- Many features may not contribute to classification.
- Noise can reduce model performance.

5. Small Sample Size

- Only 500 samples are available.
- Insufficient data to learn 10,000 feature relationships effectively.

b) Proposed Algorithm Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Purpose:

- Ensures all features are on the same scale.
- Improves optimization performance.

Step 2: Feature Selection

Methods:

- Chi-Square Test
- Information Gain
- Mutual Information
- LASSO Feature Selection

Purpose:

- Remove irrelevant features.
- Reduce dimensionality.

Step 3: Dimensionality Reduction

Use Principal Component Analysis (PCA).

PCA transforms features into fewer components while retaining maximum variance.

Example:

1000 Features

↓

PCA

↓

100 Principal Components

Benefits:

- Reduces redundancy.
- Improves computational efficiency.
- Reduces overfitting risk.

Step 4: Regularization

Use:

L1 Regularization (LASSO) $Loss + \lambda \sum |w|$

L2 Regularization (Ridge) $Loss + \lambda \sum w^2$

.

Step 5: Model Selection

Suitable algorithms:

- Support Vector Machine (SVM)
- Logistic Regression with Regularization
- Random Forest with Feature Selection

Preferred Choice:

Regularized SVM

Reasons:

- Works well with high-dimensional data.
- Effective when features exceed samples.
- Strong generalization capability.

Step 6: Cross Validation

Use:

K-Fold Cross Validation

Purpose:

- Reliable performance estimation.
- Detects overfitting.
- Improves model selection.
- .

Diagram

Dataset

(10000 Features, 500 Samples)



Data Preprocessing



Feature Selection



Dimensionality Reduction

(PCA)



Regularized Model

(SVM / Logistic)



Cross Validation



Final Classifier



Improved Generalization

Conclusion

Training a classification model with 10,000 features and only 500 samples is challenging because of the curse of dimensionality, increased computational complexity, and a high risk of overfitting. An effective strategy involves data preprocessing, feature selection, dimensionality reduction using PCA, regularization, and cross-validation. Feature selection removes irrelevant variables, PCA reduces dimensionality while preserving important information, and regularization controls model complexity. Together, these techniques reduce overfitting and improve generalization performance. Therefore, a carefully designed machine learning pipeline is essential for building accurate and robust classifiers in high-dimensional datasets.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Introduction

A Multilayer Perceptron (MLP) is a type of artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Hidden neurons use activation functions such as the sigmoid function to learn complex relationships between inputs and outputs. Increasing the number of hidden neurons can improve the network's ability to represent complex patterns, but it may also introduce challenges related to overfitting and generalization.

a) Why Adding More Hidden Neurons Increases Representational Power

A hidden neuron with sigmoid activation is given by:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Each hidden neuron learns a different feature or pattern from the input data.

When more hidden neurons are added:

- The network can learn more complex decision boundaries.
- More input-output relationships can be represented.
- The network can capture nonlinear patterns more effectively.
- Feature extraction capability increases.

b) Effect on Learning Capability

With more hidden neurons:

- Model capacity increases.
- More parameters (weights and biases) are learned.
- Complex relationships can be captured.
- Training accuracy often improves.

c) Why More Hidden Neurons Do Not Always Improve Performance

Although representational power increases, performance on unseen data may not improve.

Reasons:

1. Overfitting

- Network memorizes training data.
- Learns noise instead of useful patterns.
- High training accuracy but poor testing accuracy.

Example:

Training Accuracy = 99%

Testing Accuracy = 72%

2. Increased Complexity

- More parameters require more training data.
- Small datasets may not support large networks.

3. Longer Training Time

- More computations are required.
- Optimization becomes more difficult.

4. Higher Memory Requirements

- Large networks consume more resources.

d) Relationship Between Overfitting and Generalization

Overfitting

Occurs when:

Model Complexity > Data Complexity.

Generalization

Generalization refers to the ability of the model to perform well on unseen data.

Good generalization occurs when:

- The model captures underlying patterns.
- Noise is ignored.
- Training and testing accuracies are similar.

Diagram

Increase Hidden Neurons



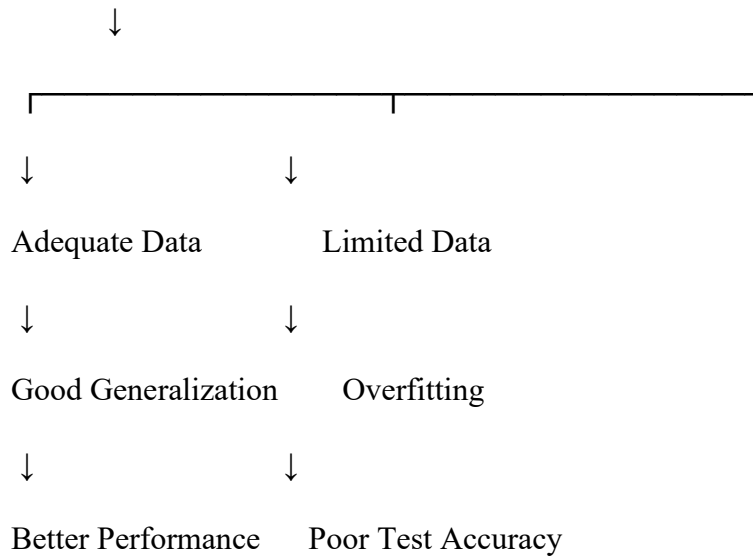
Higher Model Capacity



Better Pattern Learning



Higher Training Accuracy



Conclusion

Adding more hidden neurons to a Multilayer Perceptron increases its representational power because the network can learn more complex features and nonlinear relationships. This enhanced capacity allows the model to approximate complicated functions and achieve higher training accuracy. However, increasing the number of hidden neurons also raises the risk of overfitting, especially when training data is limited. Overfitted networks memorize training examples rather than learning general patterns, leading to poor performance on unseen data. Therefore, while additional hidden neurons improve learning capability, they do not always guarantee better performance. A balance between model complexity and generalization must be maintained using techniques such as regularization, dropout, and cross-validation to achieve optimal results.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Introduction

Deep Neural Networks (DNNs) are trained using Backpropagation and optimization algorithms such as Stochastic Gradient Descent (SGD). When the dataset contains a very large number of features, the model faces the **Curse of Dimensionality**, which affects gradient computation, convergence speed, and generalization performance. High-dimensional data increases model complexity and makes learning more challenging.

a) Impact on Gradient Updates in Backpropagation

Backpropagation updates weights using gradients:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

In high-dimensional datasets:

- Number of weights increases significantly.
- More gradients must be computed.
- Many features may be irrelevant or noisy.
- Gradients become noisy and unstable.
- Risk of vanishing or exploding gradients increases.

Effects:

- Slower learning.
- Unstable parameter updates.
- Difficulty identifying important features.

b) Impact on Convergence Speed of SGD

SGD updates weights after processing individual samples.

In high-dimensional spaces:

- Optimization landscape becomes more complex.
- More parameters require more updates.

- Gradient estimates become noisy.
- Convergence becomes slower.

c) Impact on Model Generalization

Generalization refers to performance on unseen data.

In high-dimensional datasets:

- Models may memorize training samples.
- Overfitting becomes more likely.
- Training accuracy increases.
- Test accuracy decreases

Technique 1: Dimensionality Reduction (PCA)

Principal Component Analysis (PCA) reduces the number of features while retaining most of the information.

Benefits:

- Removes redundant features.
- Reduces noise.
- Decreases computational complexity.
- Improves SGD convergence.

Technique 2: Regularization (L2 Regularization)

Regularized loss:

$$Loss + \lambda \sum w^2$$

Benefits:

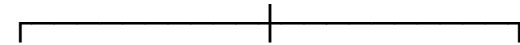
- Penalizes large weights.
- Reduces model complexity.
- Prevents overfitting.
- Improves generalization.

Diagram

High-Dimensional Dataset



Curse of Dimensionality



Backprop SGD Generalization

Issues Slowdown Problems



Noisy Slow Overfitting

Gradients Convergence



Apply Solutions



PCA / Feature Selection

Regularization / Dropout



Reduced Complexity



Faster Convergence

Conclusion

The Curse of Dimensionality significantly affects deep neural network training by making gradient updates noisy, slowing the convergence of SGD, and increasing the risk of overfitting. As dimensionality increases, the number of parameters grows, making optimization more difficult and reducing the model's ability to generalize to unseen data. Techniques such as **Principal Component Analysis (PCA)**, **feature selection**, **regularization**, and **dropout** effectively reduce model complexity and improve learning efficiency. By reducing unnecessary features and controlling model capacity, these methods enable faster convergence, more stable gradient updates, and better generalization performance, leading to more accurate and reliable deep learning models.